Internship Report

By

Vora Shahil I (22CEUBG065)

A Report Submitted

In

partial fulfillment of the requirements
for the degree of

BACHELOR OF TECHNOLOGY
In
Computer Engineering

Internal Guide
Dr. Shaifali P Malukani
Assistant Professor
Dept. of Comp. Engg.

External Guide
Ms. Yashvi Shah
Project Coordinator
RKIT

Faculty of Technology
Department of Computer Engineering
Dharmsinh Desai University

April 2026

CERTIFICATE

This is to certify that the project work titled

Sehat is the bonafide

work of

Vora Shahil I (22CEUBG065)

carried out in the partial fulfillment of the degree of Bachelor of Technology in Computer Engineering at Dharmsinh Desai University in the academic session
December 2025 to April 2026.

Dr. Shaifali P Malukani
Asst. Professor
Dept. of Computer Engg.

Dr. C. K. Bhensdadia
Head,
Dept. of Computer Engg.

Faculty of Technology
Department of Computer Engineering
Dharmsinh Desai University
April 2026

Certificate

RKIT
SOFTWARE PVT. LTD.

Date: 10/04/2026

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr. Shahil Vora**, a student of **Dharmsinh Desai University, Nadiad**, is undergoing an internship at our company, **RKIT Software Pvt. Ltd.** for the branch located at Ahmedabad as a **Full Stack Developer (Trainee)**.

During the internship period from **January 1st, 2026 to April 10th, 2026**, you have been maintaining a positive approach towards the assigned tasks.

Moreover, your apprenticeship is on-going till 30th June 2026 as per the offer letter provided by us at the time of joining.

For, RKIT Software Pvt. Ltd.

RKIT Software Pvt. Ltd. RAJKOT

(Authorized Signatory)

MIRACLE Accounting Software
RKIT Software Pvt Ltd
R K House, 4/11 - Bhaktinagar Station Plot, Rajkot 360 002. (Gujarat) INDIA
: (0281) 6161400 : sales@rkitsoftware.com : www.rkitsoftware.com

Acknowledgements

I would like to express my sincere gratitude to my company mentor at RKIT Software Pvt. Ltd., Yashvi Shah, Project Coordinator, for their constant guidance, patience, and support throughout the internship. Their practical insights into professional software development and willingness to clarify doubts at every stage made this experience both productive and enriching.

I am equally thankful to my internal college guide, Dr. Shaifali P Malukani, Department of Computer Engineering, Dharmsinh Desai University, for their academic oversight and valuable feedback during the course of this internship. Their direction helped shape the structure and quality of this report.

I would also like to extend my appreciation to my colleagues and fellow interns at RKIT Software for creating a collaborative and supportive work environment. The discussions, shared learning, and teamwork during day-to-day tasks contributed greatly to my overall growth during this period.

Finally, I am grateful to the Department of Computer Engineering, Dharmsinh Desai University, and the management of RKIT Software Pvt. Ltd. for providing this opportunity to gain real-world exposure to enterprise software development.

Abstract

This report documents the internship undertaken at **RKIT**, covering the work completed over the internship period. The internship began with a structured training phase encompassing MySQL, C#, JavaScript, jQuery, HTML, CSS, .NET Framework, .NET Core, and version control. Practical understanding was reinforced through notes, demonstrations, and assignments for each technology. Additionally, the DevExtreme frontend library was studied to understand its role in the company's UI development workflow.

Following the training phase, a fully functional backend system was developed using both .NET Framework and .NET Core, integrating the technologies studied during training. A significant research and development task involved migrating the company's internal NuGet packages to versions compatible with both .NET Framework 4.8 and .NET 10. This involved analysing breaking changes, implementing code-level modifications, debugging, and publishing the updated packages. The internship also included training on the company's internal software modules, culminating in an in-depth study of the assigned module and its business domain.

Table Of Contents

Chapter 1: Introduction..... 1
1.1 Background and Motivation..... 1
1.2 Objectives of the Internship..... 1
1.3 Report Organization..... 1
Chapter 2: Organization Profile..... 2
2.1 Company Overview..... 2
2.2 Domain and Products..... 2
2.3 Technology Stack Overview..... 2
Chapter 3: Training Phase..... 4
3.1 Technologies Studied..... 4
3.1.1 MySQL.....4
3.1.2 C#..... 4
3.1.3 JavaScript and jQuery..... 5
3.1.4 HTML, CSS and Bootstrap..... 5
3.1.5 .NET Framework..... 6
3.1.6 .NET Core..... 6
3.2 Version Control Training..... 7
3.3 Notes, Demos, and Assignments..... 7
3.4 DevExtreme Frontend Library Study..... 8
Chapter 4: R&D — NuGet Package Migration..... 10
4.1 Problem Statement..... 10
4.2 Research and Migration Strategy..... 11
4.3 Code-Level Changes..... 12
4.4 Outcome..... 13
Chapter 5: Module Training and Assigned Module..... 14
5.1 Internal Modules Overview.....14
5.2 Assigned Module — Deep Dive..... 14
Appendix: Project Documentation..... 19
A.1 Introduction..... 19
A.1.1 Overview..... 19
A.1.2 Problem Statement..... 19
A.1.3 Objectives..... 19
A.1.4 Scope..... 20
A.2 About the System..... 20
A.2.1 System Overview..... 20
A.2.2 System Architecture..... 20
A.2.3 Technology Stack..... 21
A.2.4 Key Modules..... 21

A.2.5 User Roles & Access Control..... 22
A.3 Analysis..... 23
A.3.1 ER Diagram..... 23
A.3.2 Use Case Diagram — Appointment Booking..... 24
A.3.3 Use Case Diagram — Clinic Management..... 25
A.3.4 Activity Diagram — Appointment Booking..... 26
A.3.5 Level 0 DFD..... 27
A.3.6 Level 1 DFD..... 28
A.4 Design..... 29
A.4.1 Database Design..... 29
A.4.1.1 Users Table (TBL01)..... 29
A.4.1.2 Patients Table (TBL02)..... 30
A.4.1.3 Doctors Table (TBL03)..... 31
A.4.1.4 Clinics Table (TBL05)..... 32
A.4.1.5 Doctor-Clinics Table (TBL06)..... 33
A.4.1.6 Appointment Slots Table (TBL07)..... 34
A.4.1.7 Appointments Table (TBL04)..... 35
A.4.1.8 Prescriptions Table (TBL08)..... 36
A.4.2 Frontend Interface Design..... 36
A.4.3 Validations..... 37
A.4.4 Application Navigation..... 37
A.5 Implementation..... 38
A.5.1 Authentication & JWT Implementation..... 38
A.5.2 API Controllers & Service Layer..... 38
A.5.3 Appointment Booking Workflow..... 39
A.6 Screenshots of Sehat..... 40
A.7 Conclusion and Future Extensions..... 48
A.7.1 Conclusion..... 48
A.7.2 Future Extensions..... 48
Bibliography..... 50

Chapter 1

Introduction

1.1 Background and Motivation

RKIT Software is a Rajkot-based software company known for its flagship product, Miracle Accounting Software — a powerful and user-friendly accounting solution designed for businesses of all types, accountants, and Chartered Accountants. The software helps businesses streamline accounting, billing, auditing, and e-invoicing, and serves over 100,000 satisfied clients across more than 100 business segments. The company's product is built on a robust .NET-based technology stack and caters to a wide range of industries including textiles, agriculture, retail, and finance.

Working on a widely used, real-world accounting product offered a unique chance to learn industry-standard technologies such as .NET Framework, .NET Core, and C# while understanding how large-scale desktop and web applications are architected and maintained. The company's focus on continuous product improvement and its active use of modern development practices made it an ideal environment to bridge the gap between academic knowledge and professional software engineering.

1.2 Objectives of the Internship

- To gain hands-on experience in full stack web development.
- Understand version control practices in a professional environment.
- Understand the structure and working of a real world software.

1.3 Report Organization

This report is organized as follows:

- Chapter 2 provides an overview of the organization.
- Chapter 3 covers the training phase and technologies studied.
- Chapter 4 details the NuGet package migration R&D work.
- Chapter 5 covers internal module training and the assigned module.
- The Appendix contains the project documentation.
- Bibliography

Chapter 2

Organization Profile

2.1 Company Overview

RKIT Software Pvt. Ltd. is the creation of two passionate entrepreneurs who worked to empower businesses with sustainable growth and financial confidence. The company is headquartered in Rajkot, Gujarat, and operates in the domain of accounting and financial management software. In its early days, the company focused on creating customised software to meet basic accounting needs, and over time transitioned from developing general-purpose tools to delivering specialised accounting solutions crafted for individual client requirements. Today, RKIT is powered by 150+ skilled professionals, including 20+ core experts and 10+ specialised departments in development, marketing, and technical support. The company has grown to serve over 100,000 satisfied clients across more than 100 business segments, supported by a network of 75+ active dealers.

2.2 Domain and Products

RKIT Software operates in the accounting and business management software domain. Its flagship product, Miracle Accounting Software, is designed for businesses of all types, accountants, and Chartered Accountants, helping them streamline accounting, billing, auditing, and e-invoicing. The product is offered in three tiers — Billing, Lite, and Standard — catering to businesses of varying sizes and complexity. Key capabilities include one-click e-waybill generation, one-click e-invoice, digital signature support, GST compliance, inventory management, real-time financial reports, and automated invoice sharing via WhatsApp, Email, SMS, and Telegram. The software serves a wide range of industries including textiles, agriculture, retail, auto parts, petrol, mandi, and ginning, as well as accountants and Chartered Accountants. The core problem it solves is eliminating the complexity and manual effort involved in financial management for Indian businesses, particularly around GST compliance and inventory tracking.

2.3 Technology Stack Overview

RKIT Software's development workflow is built around the Microsoft .NET ecosystem. The backend is developed using both .NET Framework and .NET Core with C# as the primary

programming language, enabling the team to maintain legacy modules while also building modern, scalable services. MySQL serves as the relational database management system for storing and querying business and transactional data. On the frontend, JavaScript and jQuery are used for dynamic UI behaviour, alongside HTML and CSS for structure and styling. The DevExtreme component library by DevExpress is used extensively for building rich, data-intensive UI components such as grids, charts, and forms — well suited for an accounting application that handles large volumes of tabular data. Version control is maintained using standard tools to manage code across teams and departments. This stack collectively supports a product that must be both feature-rich and reliable for daily business-critical use.

Chapter 3

Training Phase

3.1 Technologies Studied

3.1.1 MySQL

In this phase, I developed a strong understanding of MySQL for managing relational databases, covering both basic and advanced concepts.
I used MySQL Workbench for schema design and query execution. I learned DDL operations (CREATE, ALTER, DROP) for structure definition, DML (INSERT, UPDATE, DELETE) for data handling, and DQL with SELECT queries using WHERE, ORDER BY, GROUP BY, and HAVING for efficient data retrieval.

I worked with joins (INNER, LEFT, RIGHT), aggregate functions (SUM, COUNT, AVG, MIN, MAX), subqueries, and UNION vs UNION ALL. I also implemented stored procedures, triggers, and functions for business logic, and used views for simplifying complex queries. Backup and restore techniques and the EXPLAIN keyword were used for data safety and performance optimization.

For practice, I created demos for each topic and developed Sehat's database by designing tables, relationships, and optimized queries for real-world use.

3.1.2 C#

In this phase, I built a strong foundation in C# and the .NET ecosystem, covering fundamental and advanced concepts.
I started with basics like .NET history, program structure, and access modifiers (public, private, protected, internal), along with namespaces, libraries, and enumerations. I worked with DataTables and handled date, string, math, and file operations.

Advanced topics included abstract and sealed classes, interfaces, generics, advanced file handling, and data serialization using JSON and XML. I used lambda expressions, extension methods, and LINQ for efficient coding and data manipulation.

I also explored ORM concepts, basic security (cryptography), dynamic type usage, and database connectivity with CRUD operations.

For practice, I created demos and implemented database operations using C#, strengthening real-world application development skills.

3.1.3 JavaScript and jQuery

In this phase, I developed strong client-side scripting skills using JavaScript and jQuery to build dynamic web applications.
I learned JavaScript basics like variables, data types, functions, loops, and DOM manipulation using methods such as getElementById and querySelector. I handled events (click, change) and understood event bubbling.

I explored browser storage (localStorage, sessionStorage, cookies) and used developer tools for debugging. Advanced topics included OOP concepts, ES6 features (arrow functions, classes, modules, promises), and string, math, and date operations.

Using jQuery, I simplified DOM manipulation and event handling with selectors and methods like each(), map(), and extend(). I implemented form validation, regular expressions, callbacks, and promises.

I performed AJAX operations (GET, POST, PUT, DELETE) and worked with JSON, including serialization and deserialization.

For practice, I created demos to build interactive features and handle real-time data effectively.

3.1.4 HTML, CSS and Bootstrap

In this phase, I learned front-end development using HTML, CSS, and Bootstrap to build responsive interfaces.
I started with HTML structure, elements, and semantic tags (header, nav, article, footer), along with images, tables, and forms.

In CSS, I learned selectors, the box model, Flexbox, Grid, and responsive design using media queries and relative units.

I used Bootstrap with CDN setup, grid system, and components to quickly build responsive layouts.

For practice, I created demos and developed the Sehat web interface (landing, login, signup), applying responsive design and modern UI principles.

3.1.5 .NET Framework

In this phase, I focused on building secure and optimized APIs using .NET.
I implemented CORS, authentication, authorization, and JWT for API security, ensuring proper access control.

I handled exception management for stable APIs and explored caching (HTTP caching, Redis) for performance improvement. I also studied API versioning for backward compatibility and used filters for logging, validation, and authorization.

For documentation and testing, I used Swagger and Postman.

For practice, I created demos and developed the Sehat backend, implementing secure APIs, caching, exception handling, and well-structured endpoints.

3.1.6 .NET Core

In this phase, I gained in-depth knowledge of modern backend development using .NET Core and .NET 8.
I learned project structure, Program.cs, minimal hosting, appsettings.json, environment handling, and middleware (built-in and custom).

I implemented dependency injection with service lifetimes (Transient, Scoped, Singleton) and used routing techniques with constraints.

In API development, I worked with minimal APIs, model binding, validation, filters, and response handling, along with .NET 8 features like rate limiting and endpoint filters.

For performance, I explored AOT, trimming, benchmarking, and efficient JSON handling using System.Text.Json. I also implemented logging using ILogger, structured logging, and correlation IDs, and learned deployment on IIS and Linux.

For practice, I created demos and built a complete Sehat backend, applying dependency injection, middleware, logging, and performance optimization.

3.2 Version Control Training

In this phase, I learned version control using Git to manage code efficiently and collaborate effectively.

I started with basic commands such as git init and git clone to initialize and copy repositories. I practiced commit workflows using git add, git commit, and git push to track and upload changes.

I worked with pull and fetch operations to synchronize local and remote repositories. I also learned branching concepts, including creating, switching, and merging branches using git branch, git checkout, and git merge.

Additionally, I understood how to handle merge conflicts by identifying conflicting changes and resolving them properly.

For practical understanding, I created small demos for each topic and regularly used Git to manage my project code, ensuring proper version tracking and backup.

This phase helped me understand real-world version control practices and improved my workflow in managing and collaborating on projects.

3.3 Notes, Demos, and Assignments

The training was structured to emphasize both self-learning and practical implementation. For each topic, I created my own notes and demos, which helped in building a deeper understanding of the concepts rather than relying solely on provided material.

Regular assignments were given to reinforce learning and ensure practical application of concepts. These assignments were aligned with real-world scenarios and required proper implementation, not just theoretical knowledge.

The evaluation process included two review (reconcile) meetings per week, where I presented my demos, notes, and completed assignments. During these sessions, detailed feedback was

provided, and production-level questions were asked to assess depth of understanding, coding practices, and real-world applicability.

The overall format was highly interactive and focused on continuous improvement through hands-on demos, code reviews, and technical discussions, which significantly enhanced both my conceptual clarity and practical skills.

3.4 DevExtreme Frontend Library Study

In this phase, I performed an in-depth study of DevExtreme, as it is the primary frontend library used in my organization.

I started with the basics of DevExtreme, including installation, widget initialization, configuration, handling events, and lifecycle management (create, update, destroy). I worked extensively with various editor components such as CheckBox, DateBox, SelectBox, TextBox, NumberBox, FileUploader, RadioGroup, and implemented validation for form inputs.

I explored the data layer, including ArrayStore, CustomStore, DataSource, and LocalStore, along with query handling and filtering. This helped in understanding how DevExtreme manages and binds data efficiently.

A major focus area was the DataGrid component, where I implemented features such as paging, virtual scrolling, editing (row, batch, cell, popup, form), filtering, sorting, grouping, and selection. I also worked on advanced features like column customization, templates, state persistence, summaries, master-detail views, export functionality, and adaptability.

I integrated AJAX-based API calls with CustomStore to connect the frontend with backend services, ensuring real-time data operations.

Additionally, I implemented navigation components such as menus and tree views, and worked with dialogs and notifications including load indicators, popups, popovers, and toast messages to enhance user experience.

For practical understanding, I created small demos for each topic and also developed comprehensive demos combining multiple components. These included forms with

validation, DataGrid integrated with APIs, and UI workflows involving popups, loaders, and notifications.

This phase provided hands-on experience in building complex, scalable, and production-ready user interfaces using DevExtreme, aligning closely with real-world application development in the organization.

Chapter 4

R&D: NuGet Package Migration

4.1 Problem Statement

The existing codebase consisted of multiple internal projects targeting a mix of .NET Framework 4.8 and .NET 6. Over time, the NuGet dependencies used across these projects had become outdated due to the organization's offline package management model, where external internet access is restricted and all packages are consumed from an internally maintained repository.

This setup introduced several challenges:

- **Outdated Dependencies**
  Many packages in the offline repository were older versions and had not been updated regularly. As a result, projects were dependent on legacy APIs and lacked access to improvements, security fixes, and performance optimizations.
- **Compatibility Gaps**
  With the target migration toward .NET 10, several incompatibilities emerged:
  - APIs available in .NET Framework 4.8 were deprecated or removed in modern .NET versions.
  - Some NuGet packages did not support both frameworks simultaneously.
  - Differences in runtime behavior, especially around configuration, serialization, and dependency injection.
- **Technical Debt**
  The use of older libraries led to increased maintenance overhead and limited the ability to adopt modern development practices.
- **Business Need for Migration**
  The migration was required to:
  - Ensure long-term maintainability and supportability
  - Improve performance and scalability
  - Standardize development on modern .NET versions

- Enable future feature development without legacy constraints

4.2 Research and Migration Strategy

The migration process began with a structured research phase focused on identifying compatibility boundaries and planning a safe upgrade path.

Research Approach

- Audited all projects to extract:
  - Target frameworks
  - Current NuGet dependencies and versions
- Identified the latest compatible versions of each package that support:
  - .NET Framework 4.8 (where required)
  - .NET 10 (or .NET Core/.NET Standard compatible versions)

Tools and Techniques Used

- NuGet package metadata (dependency graphs and supported frameworks)
- Official documentation and release notes of libraries
- Build-time errors and warnings during upgrade attempts
- Trial upgrades in isolated environments

Migration Strategy

A phased and controlled approach was followed:

1. Dependency Analysis
   - Listed all NuGet packages used across projects
   - Mapped their compatibility with target frameworks
2. Offline Repository Update
   - Downloaded required package versions externally
   - Added them to the internal offline NuGet source
3. Incremental Upgrade

    - Updated packages project-by-project instead of all at once
    - Ensured build stability after each upgrade
4. Framework Alignment
    - Migrated .NET 6 projects toward .NET 10
    - Evaluated feasibility of keeping or upgrading .NET Framework 4.8 projects
5. Code Refactoring
    - Addressed breaking changes introduced by newer package versions
6. Validation
    - Built and tested after each stage to ensure no regressions

4.3 Code-Level Changes

Updating to newer package versions and frameworks required multiple code modifications across the projects.

API Replacements

- Deprecated APIs were replaced with modern equivalents
  Example:
    - Legacy configuration APIs replaced with newer configuration patterns
    - Older HTTP handling replaced with improved abstractions

Namespace Changes

- Several libraries introduced namespace restructuring
- Required updating using statements across the codebase

Method Signature Updates

- Updated method parameters and return types
- Adjusted async/await patterns where APIs became asynchronous

Configuration Changes

- Migration from older configuration formats to newer patterns:

- Transition from web.config/app.config to structured configuration models where applicable
- Changes in logging and dependency injection configuration

Dependency Injection Adjustments

- Refactored manual dependency handling to align with modern DI patterns
- Updated service registration methods

Removed/Unsupported Features

- Eliminated usage of APIs not supported in .NET 10
- Replaced with cross-platform or modern alternatives

Build and Project File Updates

- Modified project files (.csproj) to:
  - Target updated frameworks
  - Align package references with new versions

4.4 Outcome

All targeted projects were successfully upgraded with:

- Latest compatible package versions
- Required code changes implemented

Verified compatibility:

- Projects running on .NET Framework 4.8 remained stable
- Migrated projects successfully worked on .NET 10

Chapter 5

Module Training and Assigned Module

5.1 Internal Modules Overview

The organization's internal system is structured into multiple interconnected modules, each designed to handle specific business processes and operational workflows. These modules collectively support end-to-end enterprise operations, including customer management, issue tracking, authentication, API communication, and infrastructure management.

Key business areas covered by these modules include:

- Customer Relationship Management (CRM): Handling customer data, interactions, and lifecycle management
- Ticketing System: Managing internal and external issue tracking, resolution workflows, and support processes
- Authentication & Authorization: Securing access control across applications and services
- API Management: Enabling communication between different services and systems
- Product Platforms: Delivering the accounting solution via both cloud and desktop applications

During the training phase, exposure was provided to multiple core modules, including both operational workflows and technical architecture, enabling a holistic understanding of how different systems interact within the organization.

5.2 Assigned Module — Deep Dive

During the training period, I worked across several core modules rather than being restricted to a single one. This approach provided a broader understanding of the system architecture and inter-module dependencies.

Primary Focus Areas

1. Ticketing System Module

- Purpose:
  Manages issue tracking, request handling, and resolution workflows across the organization.
- Key Features:
  - Ticket creation and assignment
  - Status tracking and escalation workflows
  - User communication and updates
- Workflows:
  - Ticket lifecycle from creation → assignment → resolution → closure
- Integration:
  - Connected with CRM for customer context
  - Integrated with authentication systems for user validation

---

2. CRM Module

- Purpose:
  Maintains customer-related data and supports business interactions.
- Key Features:
  - Customer profile management
  - Interaction tracking
  - Relationship lifecycle handling
- Integration:
  - Linked with ticketing system for issue context
  - Utilized by APIs for data access across services

---

3. Authentication Module

- Purpose:
  Provides secure access control and identity management.

- Key Features:
  - User authentication and session handling
  - Role-based access control
  - Token or credential validation mechanisms
- Technical Exposure:
  - Authentication workflows
  - Security considerations in distributed systems
- Integration:
  - Central dependency for all modules requiring user validation

---

4. API Layer

- Purpose:
  Acts as a communication bridge between different modules and services.
- Key Features:
  - Standardized endpoints for data exchange
  - Request/response handling
  - Integration with internal and external systems
- Technical Exposure:
  - API structure and routing
  - Data contracts and service interactions

---

5. Cloud Miracle (Accounting Platform)

- Purpose:
  Cloud Miracle is the company's primary accounting product, delivered as a cloud-based solution.
- Key Features (Generalized):
  - Remote access to accounting functionalities

    - Centralized data management
    - Scalable and multi-user environment
- Operational Understanding:
    - How users perform accounting operations in a cloud environment
    - Interaction with backend services through APIs
- Integration:
    - Relies on authentication for secure access
    - Uses APIs to interact with financial and business logic modules

---

6. Miracle Desktop Application

- Purpose:
  Provides a desktop-based accounting solution for users who prefer local application usage.
- Key Features (Generalized):
    - User interface for managing accounting operations
    - Local execution with backend connectivity
- Operational Understanding:
    - Differences between desktop and cloud-based workflows
- Integration:
    - Connects to the same backend systems via APIs
    - Shares authentication and data services with the cloud platform

---

Overall Integration View

All modules are interconnected:

- Authentication acts as a central security layer
- APIs enable communication between modules

- CRM and Ticketing systems share business data

This interconnected design reflects a modular and service-oriented architecture.

Appendix

Project Documentation

A.1 Introduction

A.1.1 Overview

The rapid digitalization of healthcare services has created a pressing need for efficient, reliable, and scalable platforms that can bridge the gap between patients and medical professionals. Traditional appointment scheduling methods—primarily telephone-based or walk-in—are inherently inefficient, often resulting in scheduling conflicts, prolonged wait times, and poor resource utilization for both clinics and practitioners.

This project presents the design and development of a Healthcare Appointment Booking System — **Sehat -** a RESTful backend application built on the ASP.NET Core 8 framework targeting .NET 8 and React. The system provides a structured, role-based API platform that enables patients to discover doctors, browse available appointment slots, and book consultations, while empowering doctors to manage their clinic associations, define availability windows, and handle appointment decisions and prescriptions through a unified interface.

A.1.2 Problem Statement

Manual and semi-automated appointment management systems suffer from a lack of real-time data consistency, role-based access control, and structured workflows. Patients frequently encounter difficulties identifying available practitioners in their geographic vicinity, while doctors are burdened with manual scheduling tasks that detract from clinical responsibilities. Furthermore, the absence of integrated prescription management and structured appointment lifecycle tracking (Pending, Approved, Declined, and Cancelled states) limits the continuity of care.

A.1.3 Objectives

The primary objective of this project is to develop a secure, maintainable, and extensible backend system that addresses the core inefficiencies of healthcare appointment management.

Specific goals include implementing JWT-based authentication with role differentiation between patients and doctors, enabling location-aware doctor discovery, enforcing an appointment slot booking workflow with concurrent conflict prevention, and providing a complete prescription lifecycle management module. The system is designed with a clean separation of concerns through dedicated Controllers, Service layers, DTOs, and an Entity Framework Core data access layer backed by a MySQL relational database.

A.1.4 Scope

The scope of this system encompasses user registration and authentication, patient and doctor profile management, clinic creation and association, appointment slot creation and booking, appointment decision workflows, and prescription issuance and management. The system exposes well-defined RESTful API endpoints secured by bearer token authentication and enforces fine-grained role-based access control at the controller level. The current implementation focuses exclusively on backend services; frontend integration is considered outside the scope of this report.

A.2 About the system

A.2.1 System Overview

Sehat is a full-stack healthcare appointment booking platform designed to streamline the interaction between patients and doctors through a modern, responsive web application. The system eliminates the inefficiencies of manual scheduling by providing a unified digital interface where patients can discover doctors, book appointments, and access their prescriptions, while doctors can manage their clinics, availability slots, appointment decisions, and issued prescriptions — all within a single, cohesive platform.

A.2.2 System Architecture

Sehat follows a client-server architecture with a clear separation between the presentation layer and the business logic layer. The frontend is built using React.js and communicates with the backend exclusively through RESTful API calls over HTTP. The backend, developed on

ASP.NET Core 8, handles all business logic, data validation, authentication, and database operations, exposing structured JSON responses consumed by the React client.

The backend is organized into a layered architecture comprising Controllers, Service interfaces and implementations, Data Transfer Objects (DTOs), domain Models, and a centralized Entity Framework Core DbContext. This separation ensures maintainability, testability, and a clean boundary between API exposure and data access concerns. The React frontend manages application state locally and communicates role-specific views using JWT tokens stored on the client side.

A.2.3 Technology Stack

The following table summarizes the core technologies used across the Sehat platform:

Component	**Technology / Detail**
Frontend	React.js (Vite), React Router, Axios
Backend	ASP.NET Core 8 Web API (.NET 8)
Database	MySQL with Entity Framework Core (Code-First)
Authentication	JWT Bearer Tokens with role-based access (Patient / Doctor)
Password Security	BCrypt hashing via BCrypt.Net-Next
API Architecture	RESTful API with Controller → Service → Repository pattern
Middleware	Global Exception Handler, Request Logger, Correlation ID, User Context
Rate Limiting	ASP.NET Core built-in Rate Limiter (login endpoint)

A.2.4 Key Modules

Authentication & Authorization: Sehat implements a secure signup and login flow for both patients and doctors. On successful login, the backend issues a signed JWT bearer token containing the user's identity and role. Every subsequent request from the React frontend includes this token in the Authorization header, which is validated by the backend

middleware before granting access to any protected resource. The login endpoint is additionally protected by a rate limiter to mitigate brute-force attacks.

Patient Dashboard: The patient-facing interface provides a centralized dashboard from which patients can view and update their personal and medical profile (including blood group, allergies, chronic conditions, height, weight, and location), browse available doctors filtered by proximity, select appointment slots, and track all their appointments across four lifecycle statuses — Pending, Approved, Declined, and Cancelled. Patients can also access the full list of prescriptions issued to them by their doctors.

Doctor Dashboard: The doctor-facing interface allows practitioners to manage their professional profile (including specialization, license number, years of experience, bio, and consultation fee), create and associate clinics with their account, define appointment slots per clinic, and act on incoming appointment requests by approving, declining, or cancelling them with optional notes. Doctors can also view all patients who have booked appointments with them, and manage the complete lifecycle of prescriptions they issue.

Appointment Booking Flow: The appointment booking module is the core workflow of the Sehat platform. From the React frontend, a patient selects a doctor, browses that doctor's available slots at a specific clinic, and submits a booking request with a stated reason. The backend validates the slot availability in real time, marks it as booked atomically, and persists the appointment with a default status of Pending. The doctor is then able to approve or decline the request, with the updated status reflected immediately on the patient's dashboard.

Prescription Management: Once an appointment is approved, the attending doctor can issue a digital prescription directly through the platform, specifying medication name, dosage, frequency, duration, and additional instructions. Prescriptions are linked to specific appointments and are accessible to the concerned patient via their dashboard. Doctors retain the ability to update or delete prescriptions they have issued, providing full lifecycle control over the prescribing process.

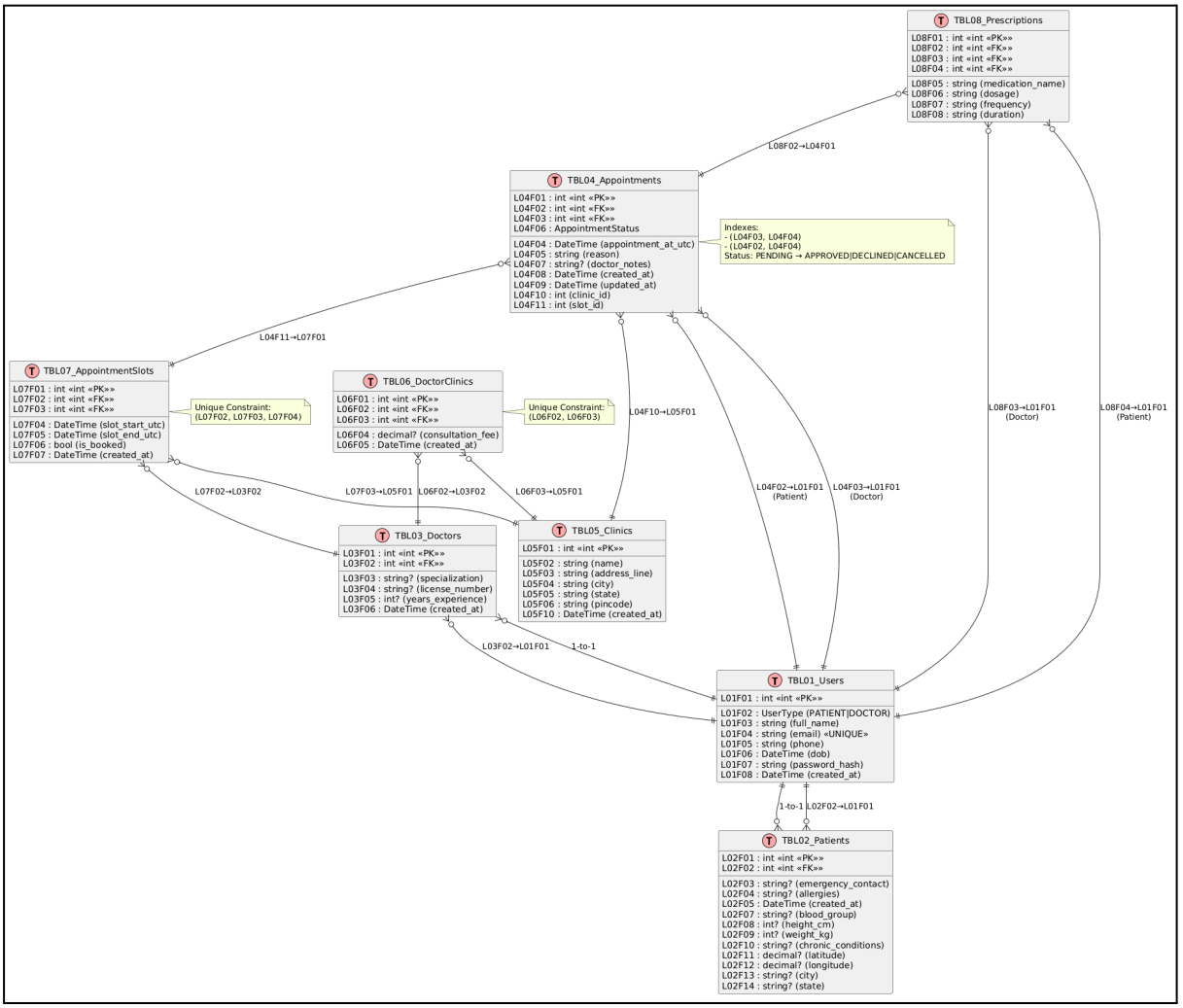
A.2.5 User Roles & Access Control

Sehat enforces a strict two-role access model. Every API endpoint on the backend verifies the authenticated user's role from the JWT token before processing the request. Patients are restricted from accessing doctor-only endpoints such as slot creation, appointment decision,

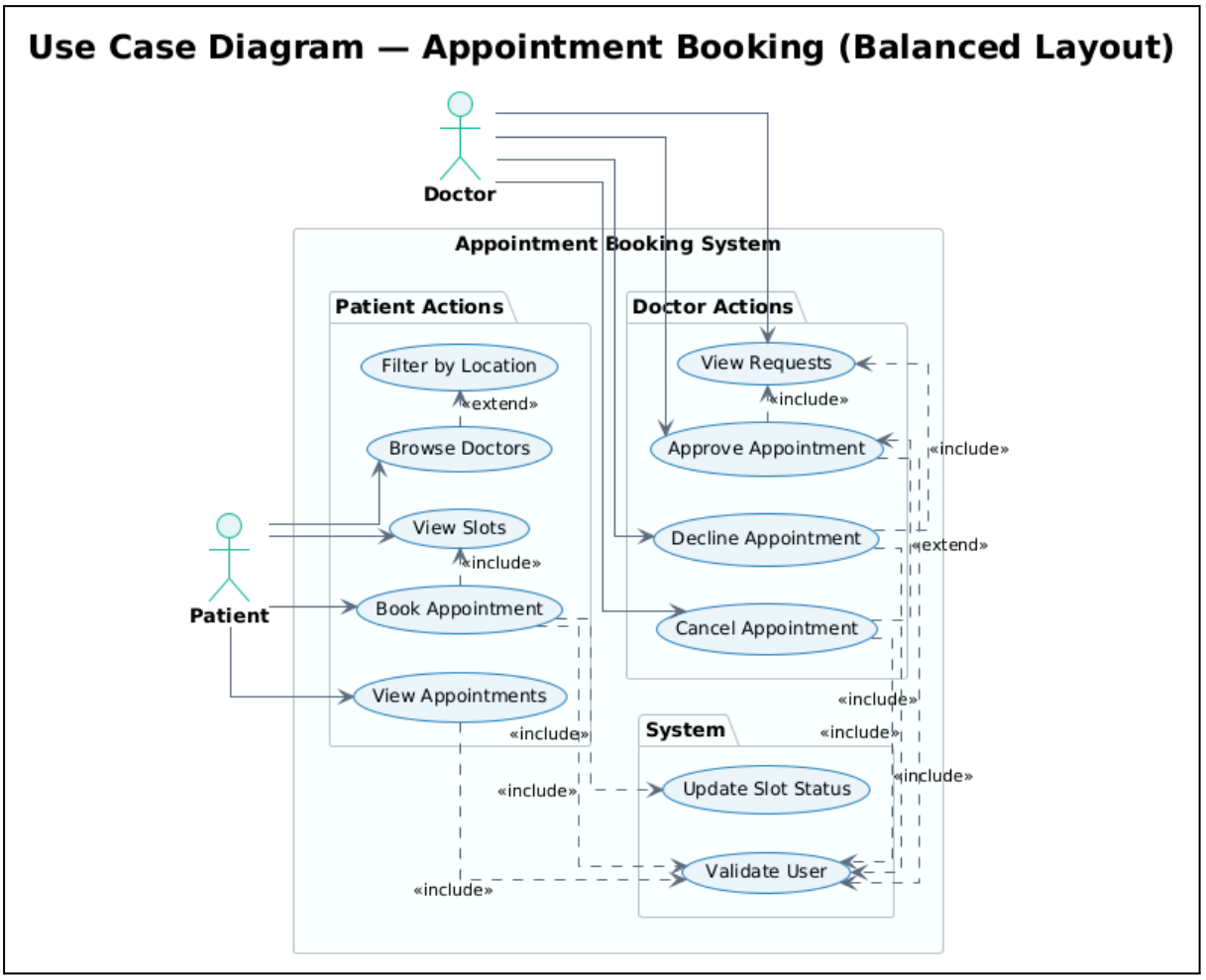
and prescription issuance, while doctors are equally restricted from patient-only endpoints such as appointment booking and doctor browsing. This role-based access control is enforced uniformly at the Controller layer, with any violation returning a 403 Forbidden response.

A.3 Analysis

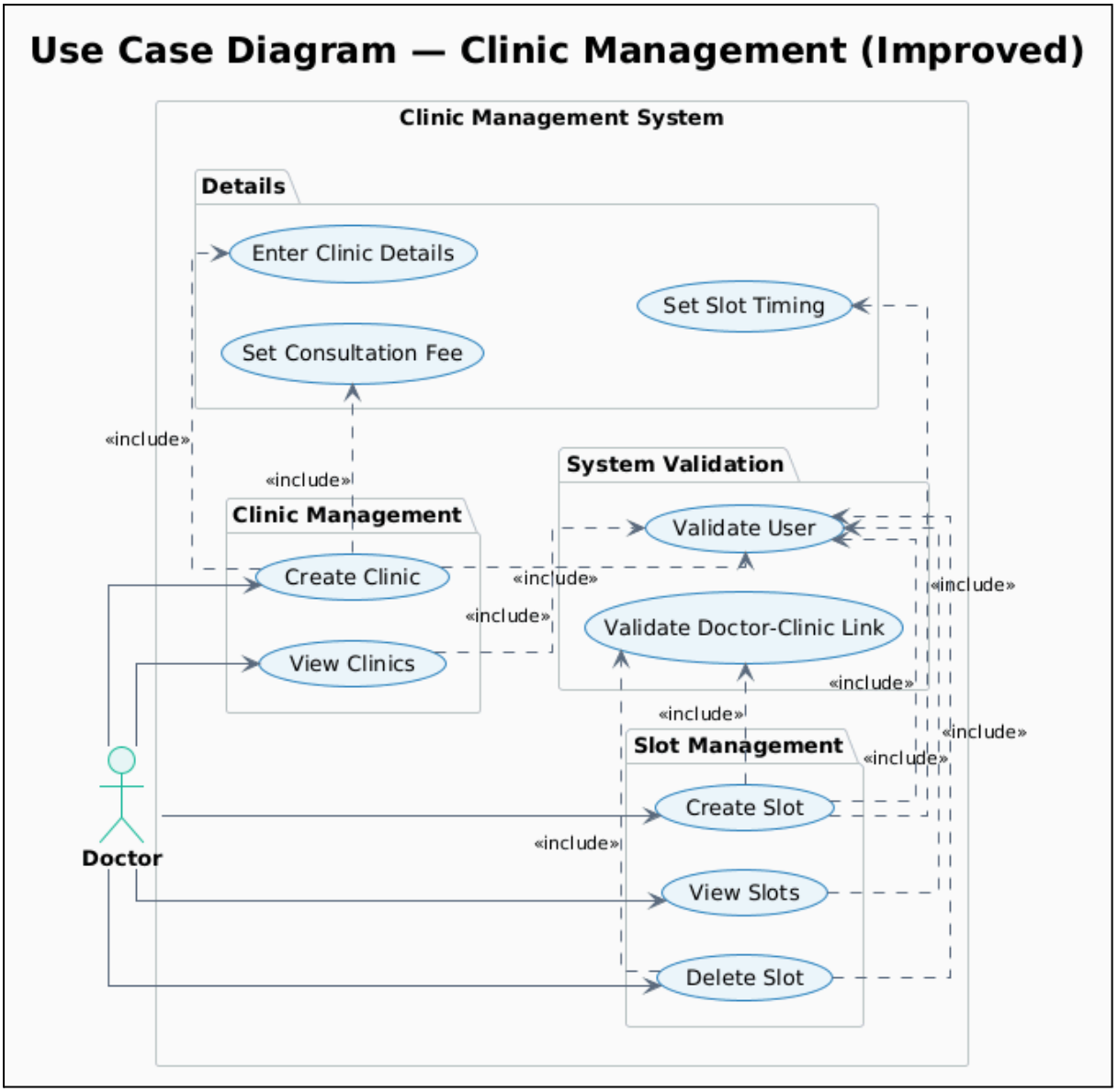
A.3.1 ER Diagram



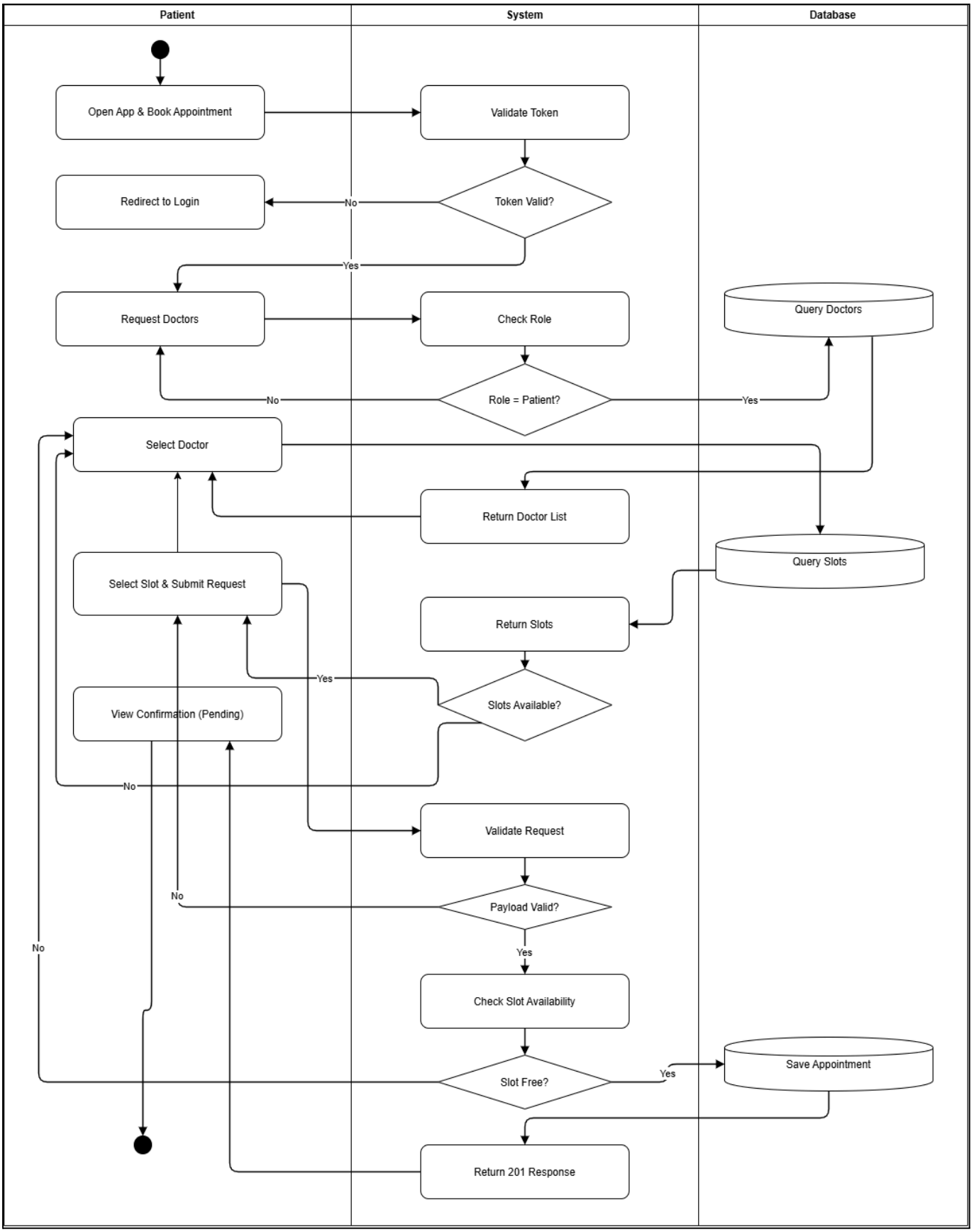
A.3.2 Use Case Diagram - Appointment Booking



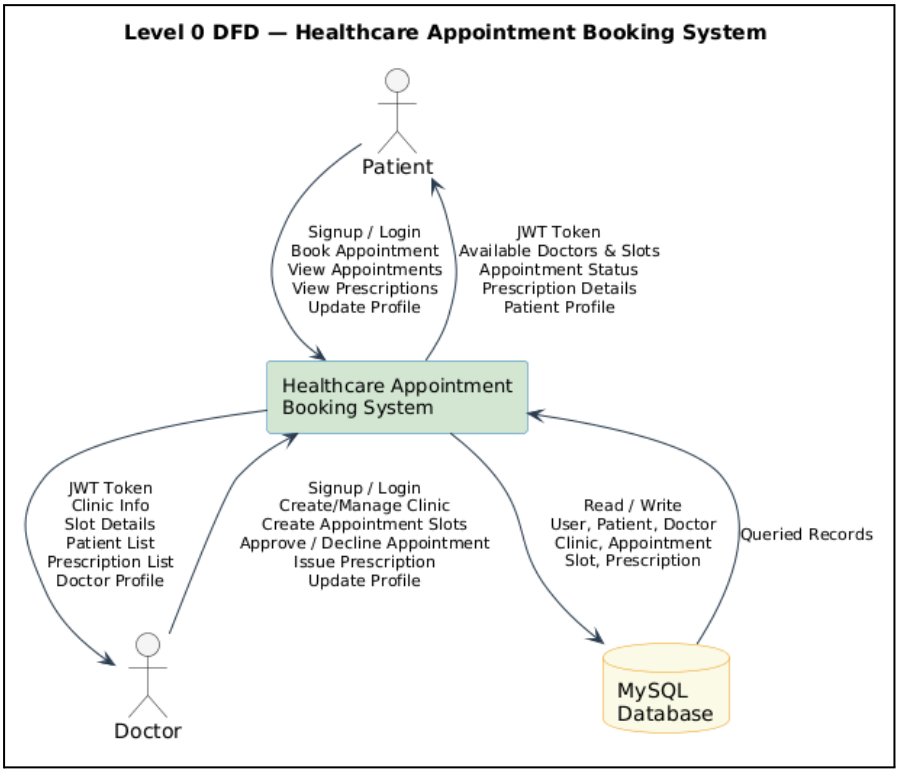
A.3.3 Use case diagram - Clinic Management



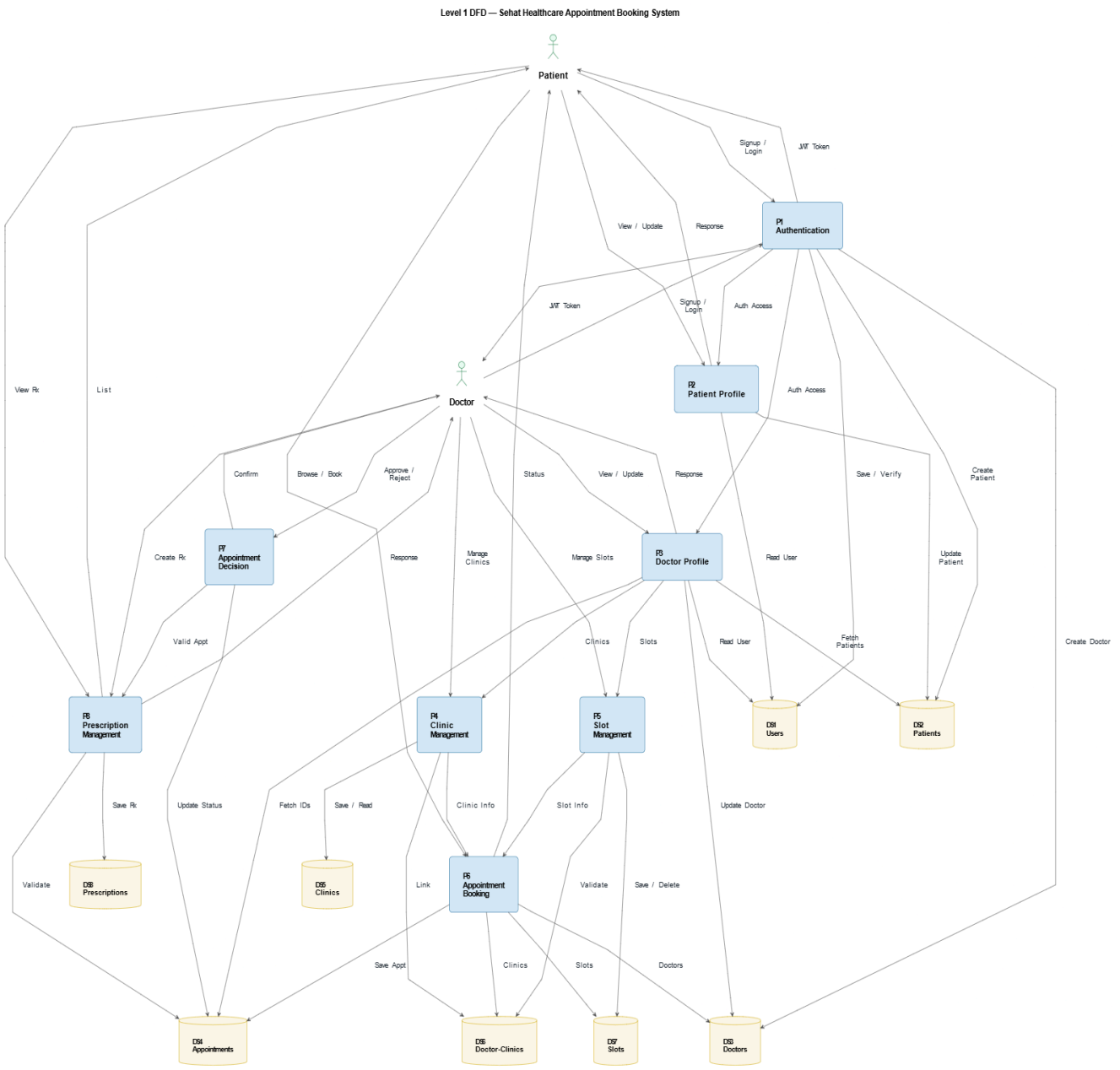
A.3.4 Activity Diagram - AppointmentBooking



A.3.5 Level 0 DFD



A.3.6 Level 1 DFD



A.4 Design

Sehat's design consists of four main components: the relational database schema, React-based frontend, validation mechanisms, and application navigation. These together define data storage, user interaction, data integrity, and system flow.

A.4.1 Database Design

The Sehat database is a relational MySQL schema managed through Entity Framework Core using the Code-First migration approach. The schema comprises eight tables, each serving a clearly defined purpose within the domain model. Relationships between tables are enforced through foreign key constraints with cascading delete behavior where appropriate. The following subsections describe each table and its role in the system.

A.4.1.1 Users Table (TBL01)

The Users table serves as the master identity record for all system users, regardless of role. It stores authentication credentials and common personal attributes shared by both patients and doctors.

Column	Data Type	Constraint	Description
id	INT	PK, Auto Increment	Unique user identifier
user_type	VARCHAR	NOT NULL	Role: PATIENT or DOCTOR
full_name	VARCHAR	NOT NULL	User's full name
email	VARCHAR	UNIQUE, NOT NULL	Login email address
phone	VARCHAR	NOT NULL	Contact phone number
dob	DATETIME	NOT NULL	Date of birth
password	VARCHAR	NOT NULL	BCrypt hashed password
created_at	DATETIME	NOT NULL	Account creation timestamp (UTC)

A.4.1.2 Patients Table (TBL02)

The Patients table extends the Users table with patient-specific medical and location information. It maintains a one-to-one relationship with the Users table via a foreign key on user_id.

Column	Data Type	Constraint	Description
id	INT	PK, Auto Increment	Patient record identifier
user_id	INT	FK → users.id	Reference to master user
emergency_contact	VARCHAR	NULLABLE	Emergency contact number
allergies	TEXT	NULLABLE	Known allergies
blood_group	VARCHAR	NULLABLE	Blood group (e.g. A+)
height_cm	INT	NULLABLE	Height in centimetres
weight_kg	INT	NULLABLE	Weight in kilograms
chronic_conditions	TEXT	NULLABLE	Chronic illness notes
latitude	DECIMAL	NULLABLE	Patient geographic latitude
longitude	DECIMAL	NULLABLE	Patient geographic longitude
city	VARCHAR	NULLABLE	Patient city
state	VARCHAR	NULLABLE	Patient state / province
created_at	DATETIME	NOT NULL	Record creation timestamp (UTC)

A.4.1.3 Doctors Table (TBL03)

The Doctors table stores professional information specific to doctor accounts. It also maintains a one-to-one relationship with the Users table and holds aggregated rating data.

Column	Data Type	Constraint	Description
id	INT	PK, Auto Increment	Doctor record identifier
user_id	INT	FK → users.id	Reference to master user
specialization	VARCHAR	NULLABLE	Medical specialization
license_number	VARCHAR	NULLABLE	Medical license number
years_experience	INT	NULLABLE	Years of practice
bio	TEXT	NOT NULL	Doctor biography
consultation_fee	DECIMAL	NOT NULL	Standard consultation fee
rating_avg	DECIMAL	NOT NULL	Average patient rating
total_reviews	INT	NOT NULL	Total number of reviews
created_at	DATETIME	NOT NULL	Record creation timestamp (UTC)

A.4.1.4 Clinics Table (TBL05)

The Clinics table stores master clinic records with full address and geographic coordinate information. A clinic is created by a doctor and linked through the Doctor-Clinics mapping table.

Column	Data Type	Constraint	Description
id	INT	PK, Auto Increment	Clinic identifier
name	VARCHAR	NOT NULL	Clinic name
address_line	VARCHAR	NOT NULL	Street address
city	VARCHAR	NOT NULL	City name
state	VARCHAR	NOT NULL	State / province
pincode	VARCHAR	NOT NULL	Postal pincode
latitude	DECIMAL	NULLABLE	Geographic latitude
longitude	DECIMAL	NULLABLE	Geographic longitude
phone	VARCHAR	NULLABLE	Clinic contact number
created_at	DATETIME	NOT NULL	Record creation timestamp (UTC)

A.4.1.5 Doctor-Clinics Table (TBL06)

The Doctor-Clinics table represents the many-to-many association between doctors and clinics, allowing a single doctor to practice at multiple clinics. A unique composite index on (doctor_user_id, clinic_id) prevents duplicate associations.

Column	Data Type	Constraint	Description
id	INT	PK, Auto Increment	Mapping identifier
doctor_user_id	INT	FK → doctors.user_id	Linked doctor
clinic_id	INT	FK → clinics.id	Linked clinic
consultation_fee	DECIMAL	NULLABLE	Fee at this specific clinic
created_at	DATETIME	NOT NULL	Record creation timestamp (UTC)

A.4.1.6 Appointment Slots Table (TBL07)

The Appointment Slots table stores time windows defined by doctors at specific clinics. A unique composite index on (doctor_user_id, clinic_id, slot_start_utc) prevents duplicate slot entries. The is_booked flag is atomically set to true when a patient books the slot.

Column	Data Type	Constraint	Description
id	INT	PK, Auto Increment	Slot identifier
doctor_user_id	INT	FK → doctors.user_id	Owning doctor
clinic_id	INT	FK → clinics.id	Clinic for this slot
slot_start_utc	DATETIME	NOT NULL	Slot start time (UTC)
slot_end_utc	DATETIME	NOT NULL	Slot end time (UTC)
is_booked	BOOLEAN	NOT NULL, Default: false	Booking status flag
created_at	DATETIME	NOT NULL	Record creation timestamp (UTC)

A.4.1.7 Appointments Table (TBL04)

The Appointments table records all booking transactions between patients and doctors. It tracks the full lifecycle of each appointment through a status field that transitions between PENDING, APPROVED, DECLINED, and CANCELLED states.

Column	Data Type	Constraint	Description
id	INT	PK, Auto Increment	Appointment identifier
patient_user_id	INT	FK → users.id	Booking patient
doctor_user_id	INT	FK → users.id	Target doctor
clinic_id	INT	NOT NULL	Associated clinic
slot_id	INT	NOT NULL	Booked slot reference
appointment_at_utc	DATETIME	NOT NULL	Scheduled date and time (UTC)
reason	VARCHAR	NOT NULL	Patient-stated reason
status	VARCHAR	NOT NULL	PENDING / APPROVED / DECLINED / CANCELLED
doctor_notes	TEXT	NULLABLE	Doctor's decision notes
created_at	DATETIME	NOT NULL	Creation timestamp (UTC)
updated_at	DATETIME	NOT NULL	Last update timestamp (UTC)

A.4.1.8 Prescriptions Table (TBL08)

The Prescriptions table stores medication details issued by a doctor for a specific appointment. Each prescription is linked to exactly one appointment and carries full medication metadata.

Column	Data Type	Constraint	Description
id	INT	PK, Auto Increment	Prescription identifier
appointment_id	INT	FK → appointments.id	Related appointment
doctor_user_id	INT	NOT NULL	Issuing doctor
patient_user_id	INT	NOT NULL	Target patient
medication_name	VARCHAR	NOT NULL	Name of medication
dosage	VARCHAR	NOT NULL	Dosage (e.g. 500mg)
frequency	VARCHAR	NOT NULL	Frequency (e.g. Twice daily)
duration	VARCHAR	NOT NULL	Duration (e.g. 7 days)
instructions	TEXT	NULLABLE	Additional instructions
issued_at_utc	DATETIME	NOT NULL	Issue date (UTC)
created_at_utc	DATETIME	NOT NULL	Record creation timestamp (UTC)

A.4.2 Frontend Interface Design

The frontend is built with React.js, focusing on role-based UI and simplicity.

- Authentication Pages: Minimal login/signup forms with role-based dynamic fields and validation.
- Patient Dashboard: Central hub for booking, tracking appointments, viewing prescriptions, and managing profile.

- Doctor Dashboard: Focused on clinic management, slot creation, appointments, and prescriptions.
- Appointment Booking Flow: Multi-step process—select doctor → choose clinic/slot → confirm booking.
- Location Selection: Uses Leaflet.js for map-based location picking (latitude/longitude).
- Prescription & Profile Views: Structured display of prescriptions and editable user profiles.

A.4.3 Validations

Sehat uses both frontend and backend validation for data integrity.

Frontend:

- Email format, password strength, required fields
- Date/time and numeric validations
- Role-based field rendering
- Map location selection validation

Backend:

- JWT authentication and role authorization
- Duplicate email prevention
- Slot availability re-check (handles concurrency)
- Doctor-clinic relationship validation
- Appointment ownership checks
- Unique slot constraint enforcement
- Global exception handling and rate limiting

A.4.4 Application Navigation

Navigation is handled using React Router with strict role-based access control.

- Public Routes: /login, /signup
- Patient Routes: Dashboard, booking, appointments, prescriptions, profile
- Doctor Routes: Dashboard, clinics, slots, appointments, patients, prescriptions, profile

- Access Control: Users can only access routes matching their role; unauthorized access redirects to the correct dashboard.

A.5 Implementation

This section covers three core areas: Authentication & JWT, Controller–Service architecture, and Appointment Booking workflow, with implementation highlights.

A.5.1 Authentication & JWT Implementation

Signup:
A unified Signup API supports both PATIENT and DOCTOR roles. Validation is handled using Data Annotations and custom logic:

- Doctors must provide specialization, license, and experience
- Patient payload removes doctor-specific fields
- Passwords are securely hashed using BCrypt
- Workflow follows: PreSave → Validate → Save

Login & JWT:

- Credentials are verified using BCrypt
- A signed JWT token (user ID + role) is generated on success
- Token is stored on the client for authenticated requests
- Login is protected with rate limiting to prevent brute-force attacks

User Context Middleware:

- Extracts user ID and role from JWT
- Stores it in a strongly typed object (CurrentUserContext)
- Accessible across controllers without re-parsing token

A.5.2 API Controllers & Service Layer

Architecture:

- Controllers handle HTTP requests and role checks

- Services handle business logic, validation, and database operations

Role Enforcement:

- [Authorize] ensures only authenticated access
- Role is validated per request; unauthorized access returns 403

Three-Step Workflow:
All write operations follow:

1. PreSave → Data preparation (e.g., hashing, mapping)
2. Validate → Business rule checks
3. Save → Database persistence

Global Exception Middleware:

- Handles all exceptions centrally
- Returns structured JSON errors
- Logs errors using NLog
- Prevents exposure of internal stack traces

A.5.3 Appointment Booking Workflow

1. Browse Doctors & Slots:

- Patients fetch available doctors (excluding already booked ones)
- Slots are filtered by doctor, clinic, and availability (`is_booked = false`)

2. Create Appointment:

- Requires doctor ID, slot ID, and reason
- Input validated via DTO annotations

3. Race Condition Handling:

- Slot availability is re-checked before booking
- If already booked → request is rejected
- Slot is atomically marked booked during save
- Appointment created with status PENDING

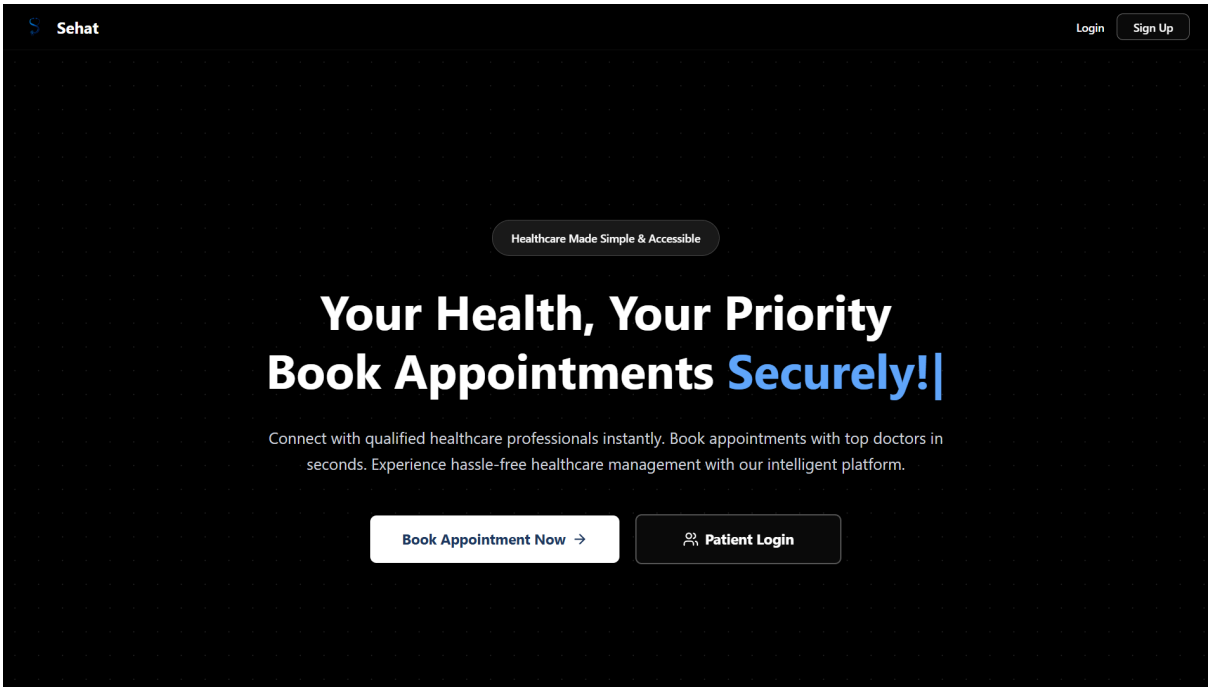
4. Doctor Decision:

- Doctors can Approve / Decline / Cancel appointments
- Validation ensures:
    - Doctor owns the appointment
    - Status transitions are valid

5. View Appointments:

- Both roles can fetch appointments by status:
  Pending, Approved, Declined, Cancelled
- Filtering is role-based (patient vs doctor)

A.6 Screenshots of Sehat



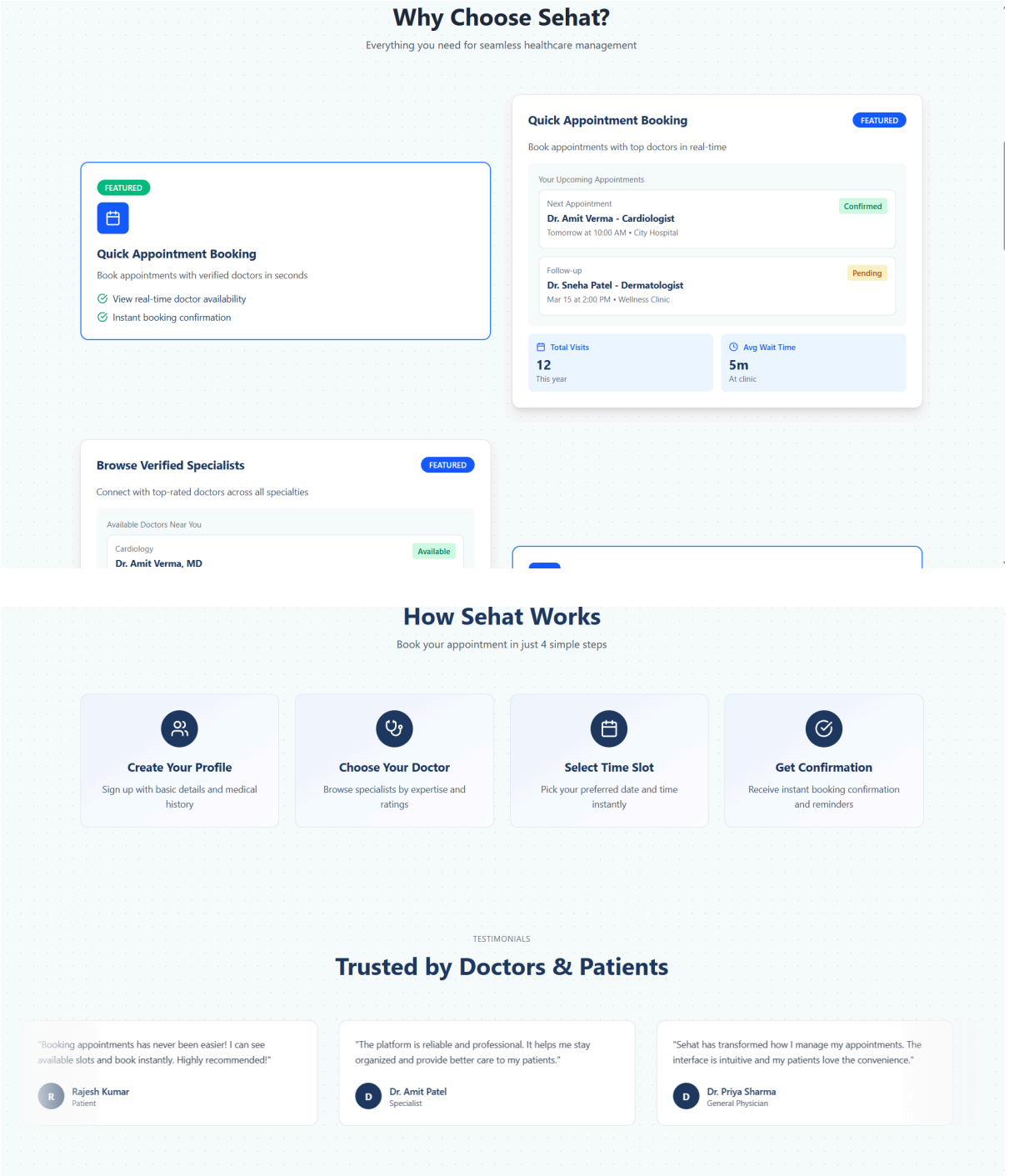


Fig A.6.1 Landing page

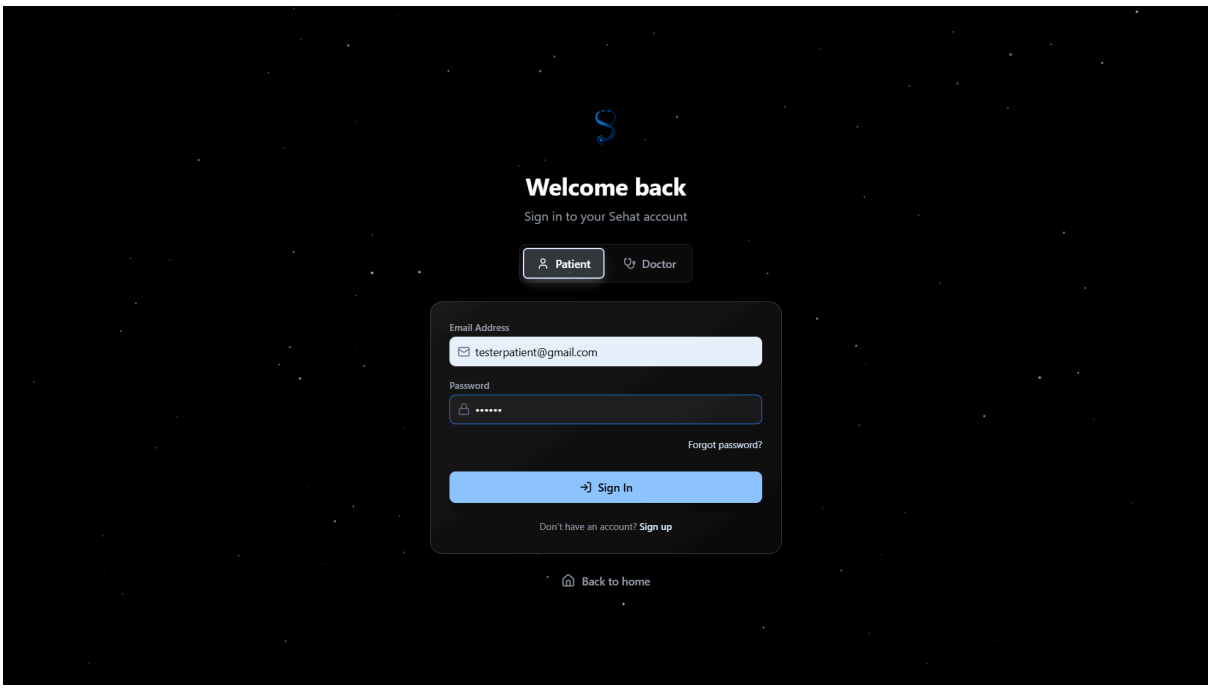


Fig A.6.2 Login Page

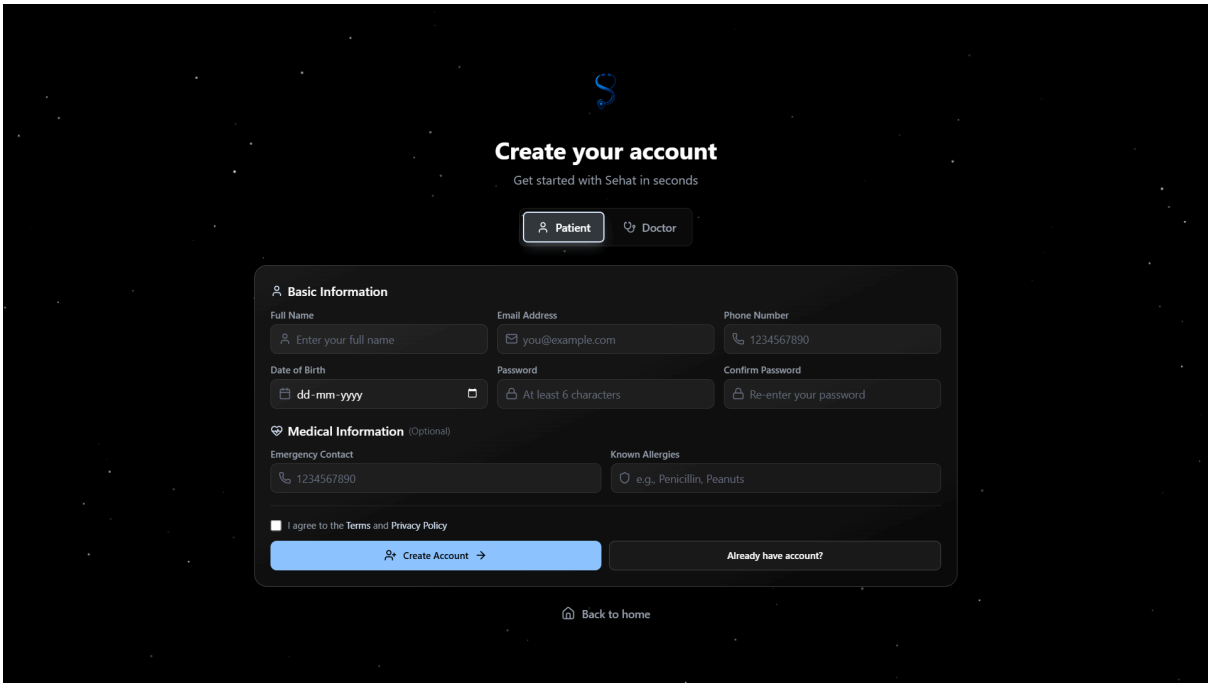


Fig A.6.3 SignupPage

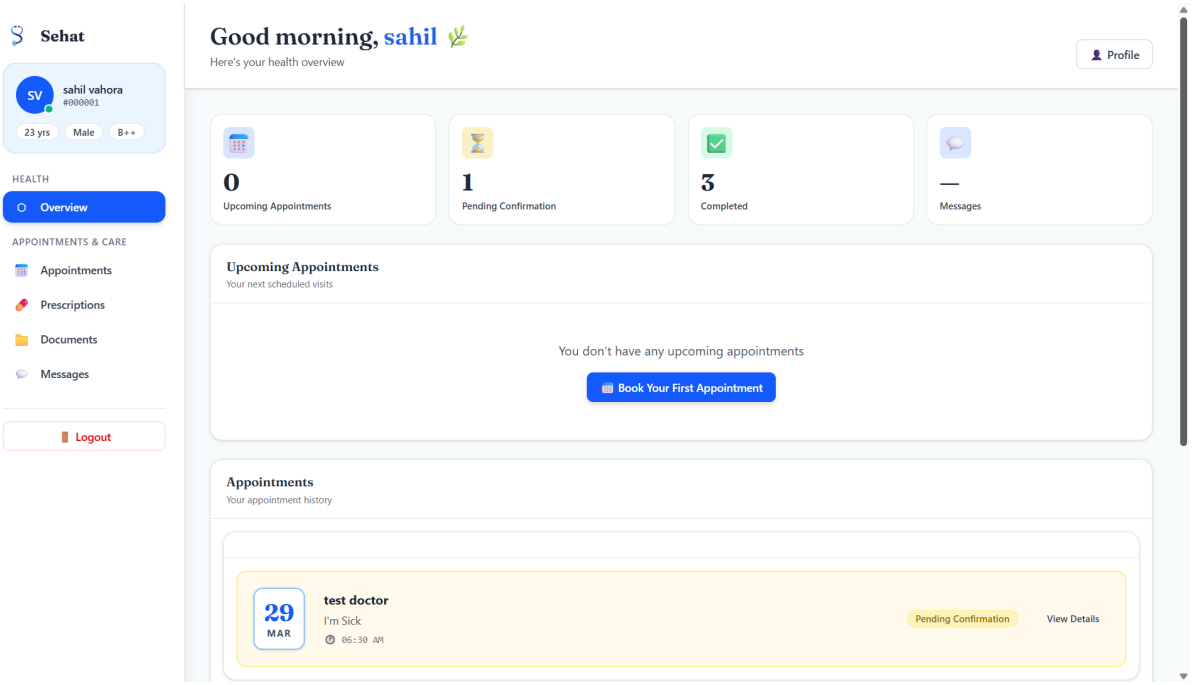


Fig A.6.4 Patient Dashboard

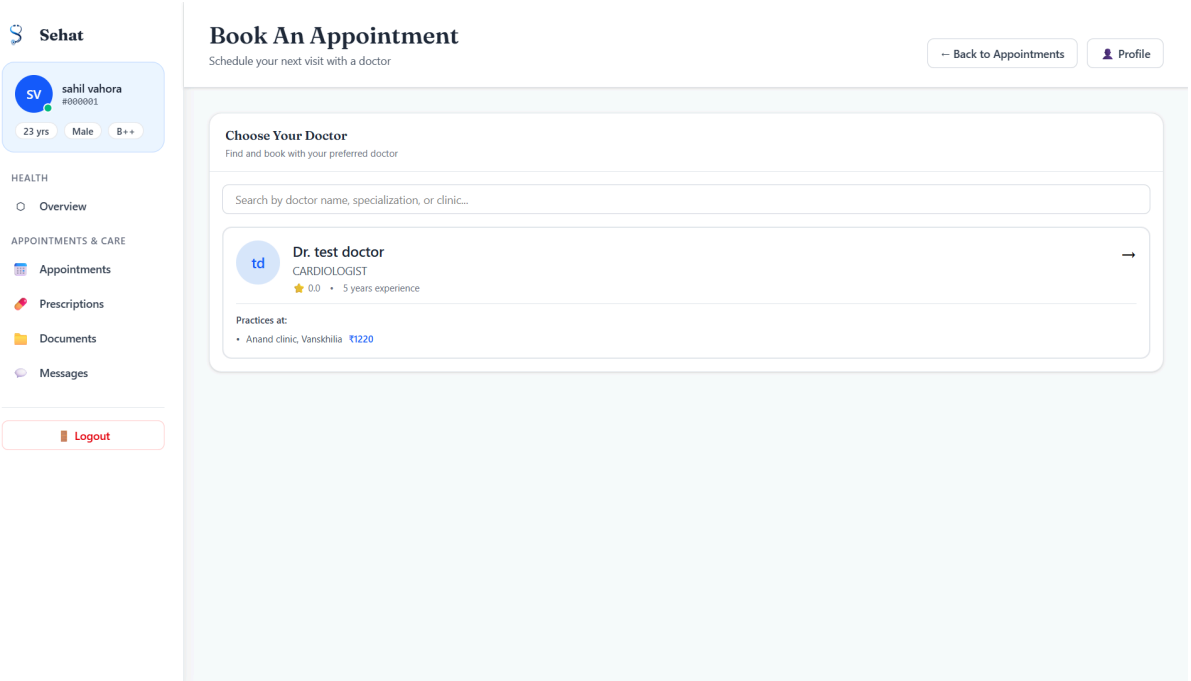


Fig A.6.5 Patient Appointment Booking Page

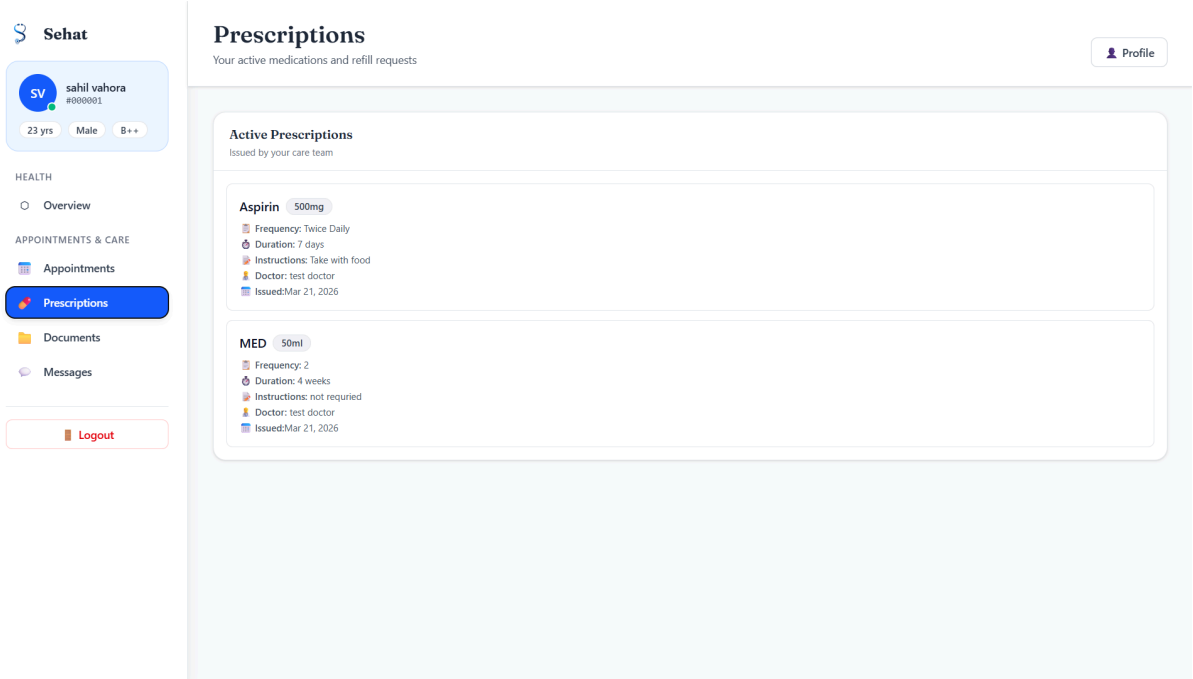


Fig A.6.5 Patient Prescriptions Page

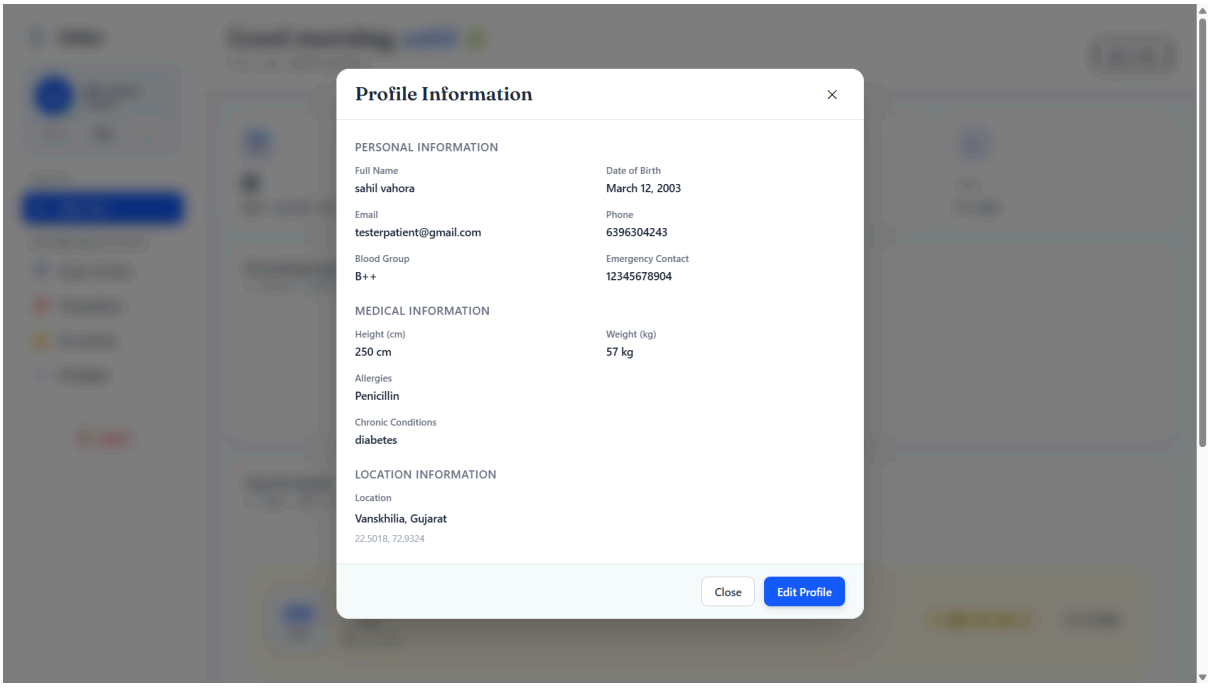


Fig A.6.6 Patient Profile

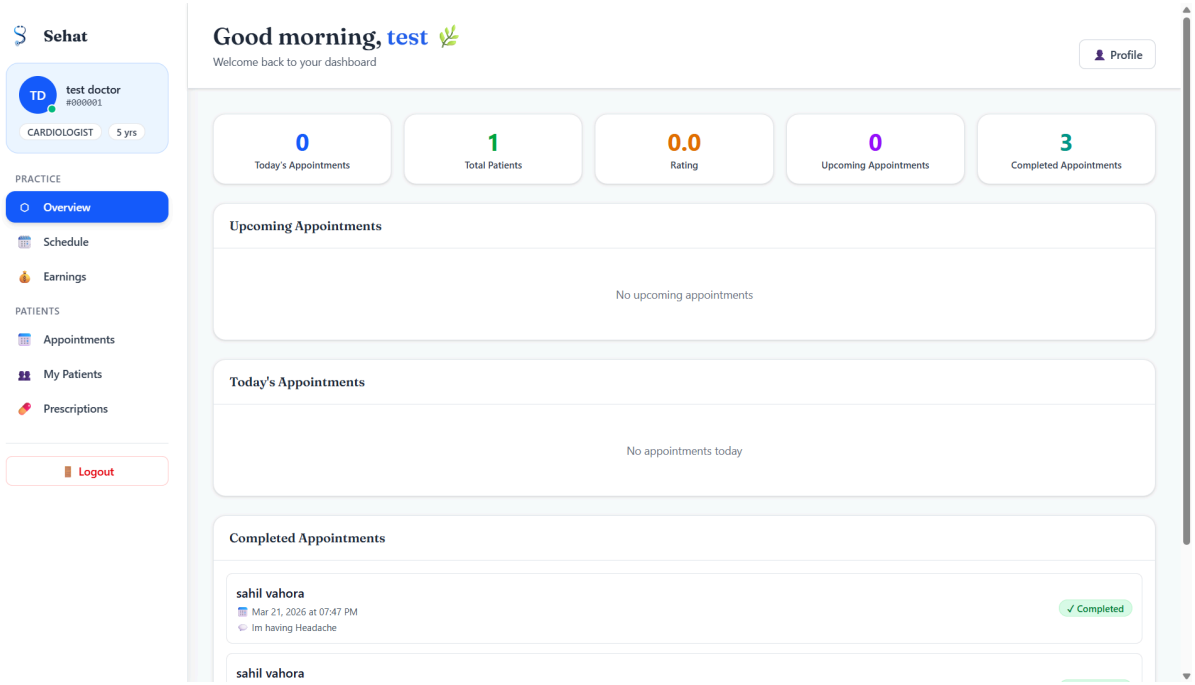


Fig A.6.7 Doctor Dashboard

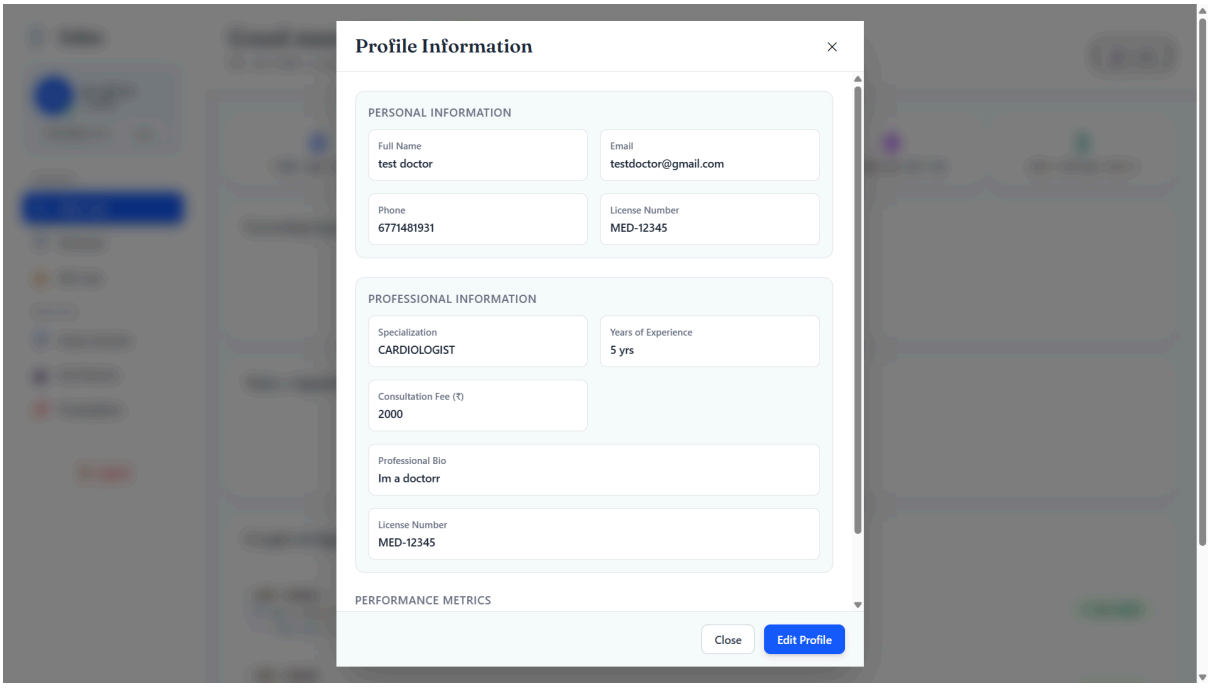


Fig A.6.7 Doctor Profile Page

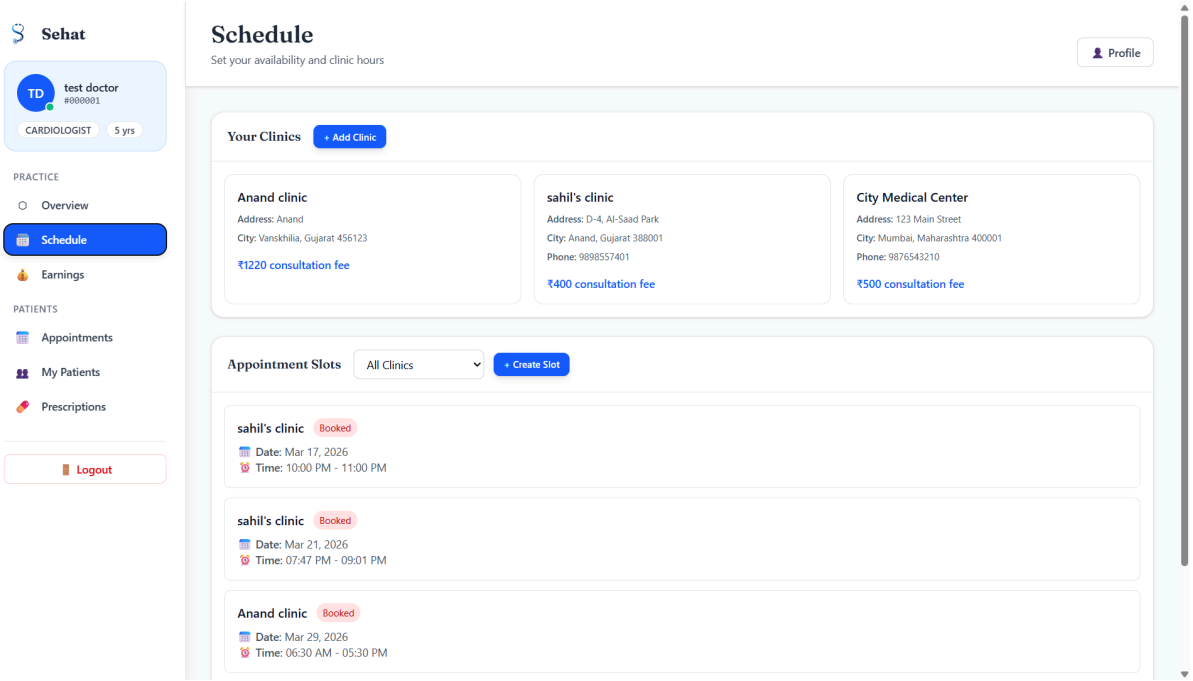


Fig A.6.8 Doctor Clinic Management Page

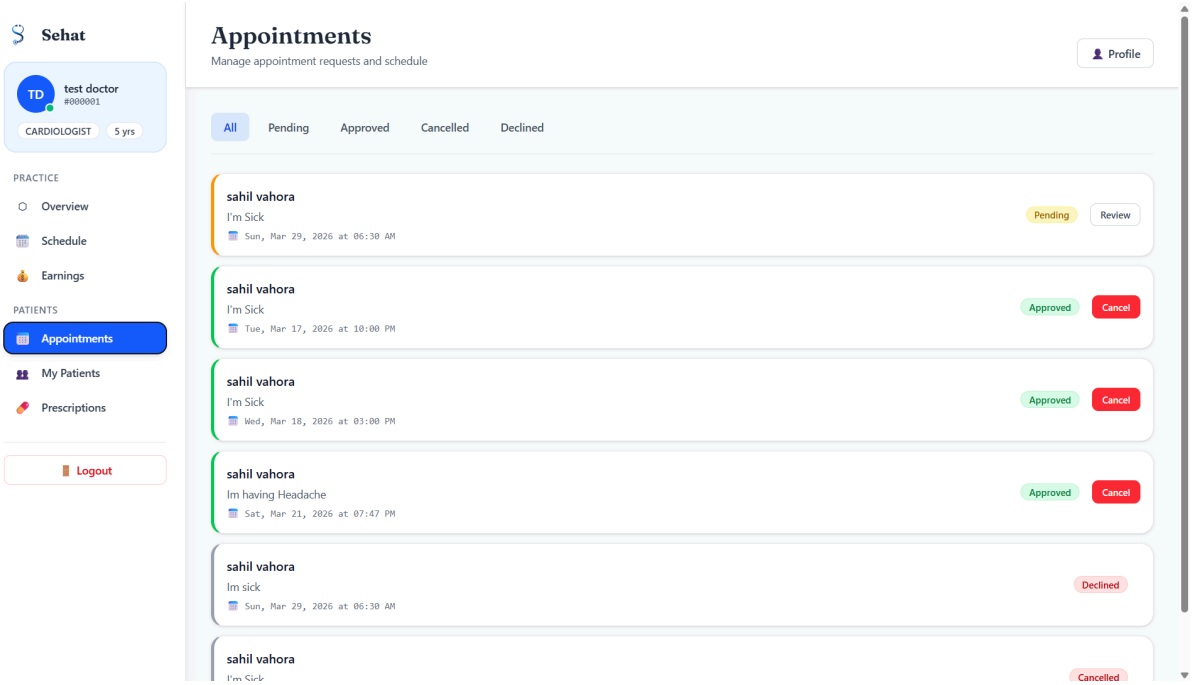


Fig A.6.9 Doctor Appointment Management Page

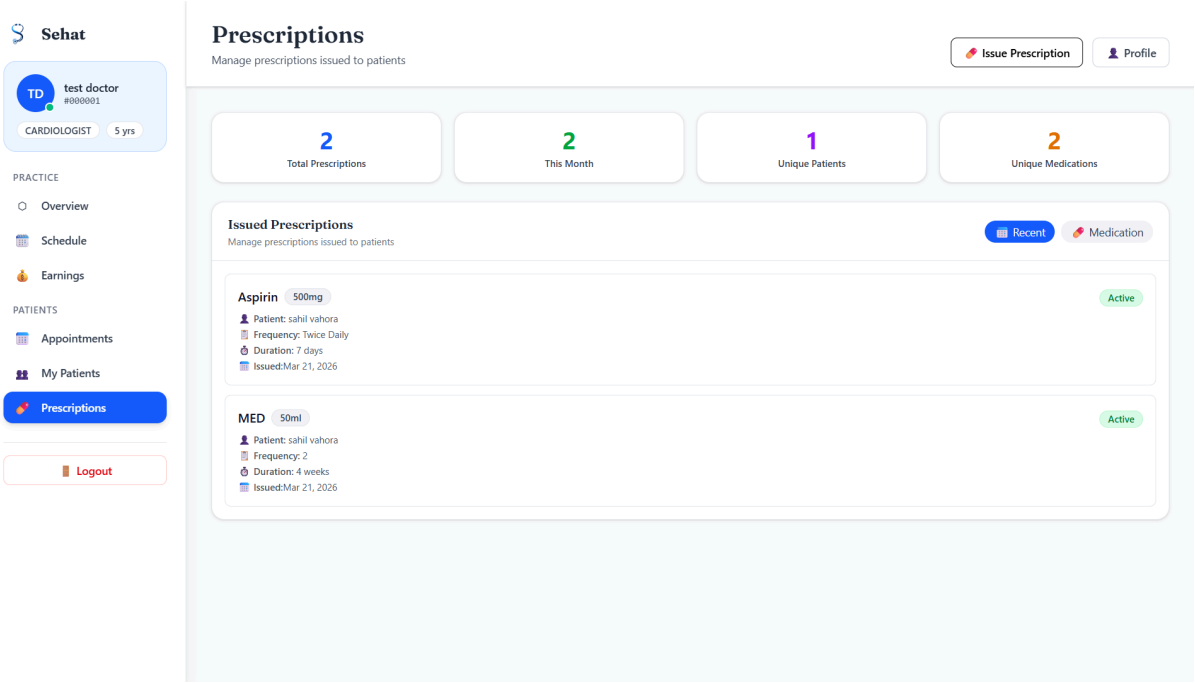


Fig A.6.10 Doctor Prescription Page

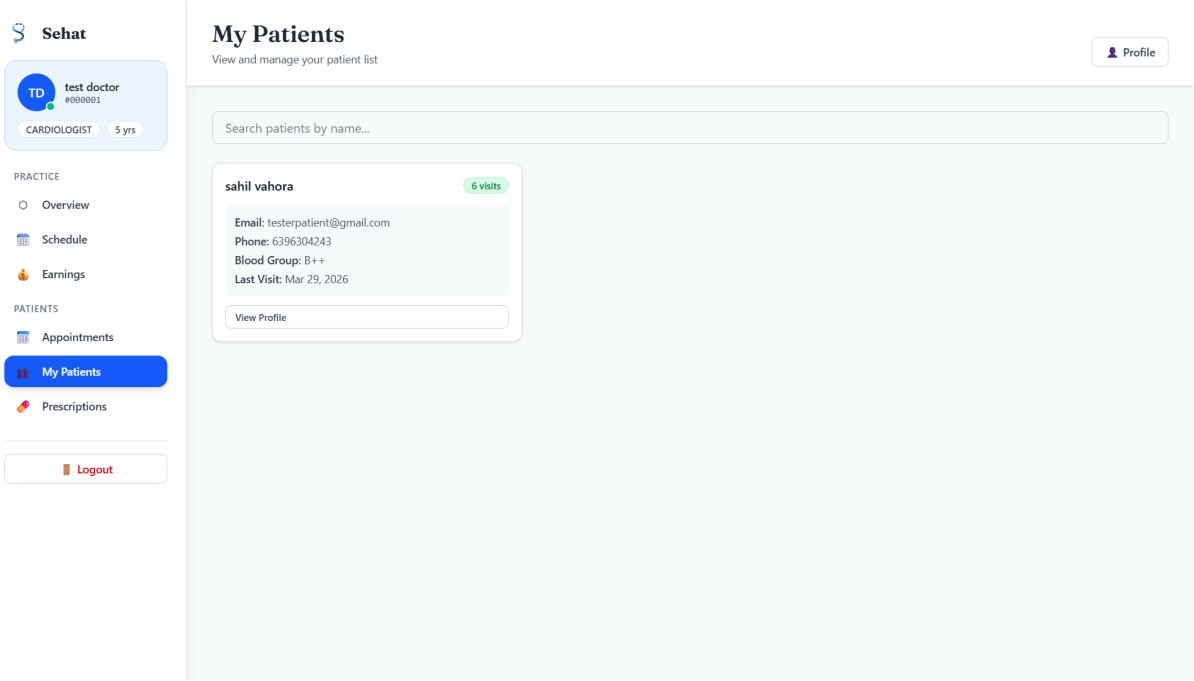


Fig A.6.11 Doctor Patient Page

A.7 Conclusion and Future extensions

A.7.1 Conclusion

Sehat was developed with the goal of digitizing and streamlining the healthcare appointment process for both patients and medical practitioners. The system successfully delivers a full-stack solution — a React.js frontend paired with an ASP.NET Core 8 RESTful backend — that covers the complete lifecycle of a healthcare interaction: user registration and authentication, clinic and slot management, appointment booking and decision workflows, and prescription management.

The implementation demonstrates a clean architectural approach through role-based access control, a layered service pattern, DTO-driven validation, and a robust middleware pipeline for exception handling, request logging, and security. The use of Entity Framework Core with a Code-First MySQL schema ensures a maintainable and evolvable data layer, while Leaflet.js integration brings geographic context to doctor discovery and clinic management.

Overall, Sehat achieves its primary objectives and provides a solid, extensible foundation upon which more advanced healthcare features can be built in future iterations.

A.7.2 Future Extensions

While the current version of Sehat addresses the core appointment booking workflow, several meaningful enhancements are identified for future development:

- **AI Health Diagnostic Agent —** A conversational AI agent will be integrated directly into the patient interface, allowing patients to describe their symptoms and receive a preliminary assessment of possible conditions before booking an appointment. Built on a large language model, the agent will guide patients towards the most relevant medical specialization, reducing misdirected bookings and helping patients make more informed decisions about the urgency and type of care they require.
- **Video Consultation Support —** An integrated video calling module will enable doctors to conduct remote consultations directly within the Sehat platform,

making the system suitable for telemedicine use cases without requiring patients to switch to a third-party application.

- **Payment Gateway Integration —** A payment module will be added to allow patients to pay consultation fees online at the time of booking. This will include support for UPI, cards, and net banking, with digital receipts generated for each transaction.

Bibliography

[1] Microsoft, "ASP.NET Core documentation," Microsoft Docs. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core.

[2] Microsoft, "Entity Framework Core — Overview," Microsoft Docs. [Online]. Available: https://learn.microsoft.com/en-us/ef/core.

[3] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519.

[4] Meta Open Source, "React — A JavaScript library for building user interfaces," [Online]. Available: https://react.dev.

[5] Remix Software, "React Router documentation," [Online]. Available: https://reactrouter.com.

[6] V. Agafonkin, "Leaflet — an open-source JavaScript library for mobile-friendly interactive maps," [Online]. Available: https://leafletjs.com.

[7] Oracle Corporation, "MySQL 8.0 Reference Manual," [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en.

[8] NLog Project, "NLog — flexible and free logging platform for .NET," [Online]. Available: https://nlog-project.org.

[9] Axios Contributors, "Axios — Promise based HTTP client for the browser and node.js," [Online]. Available: https://axios-http.com.